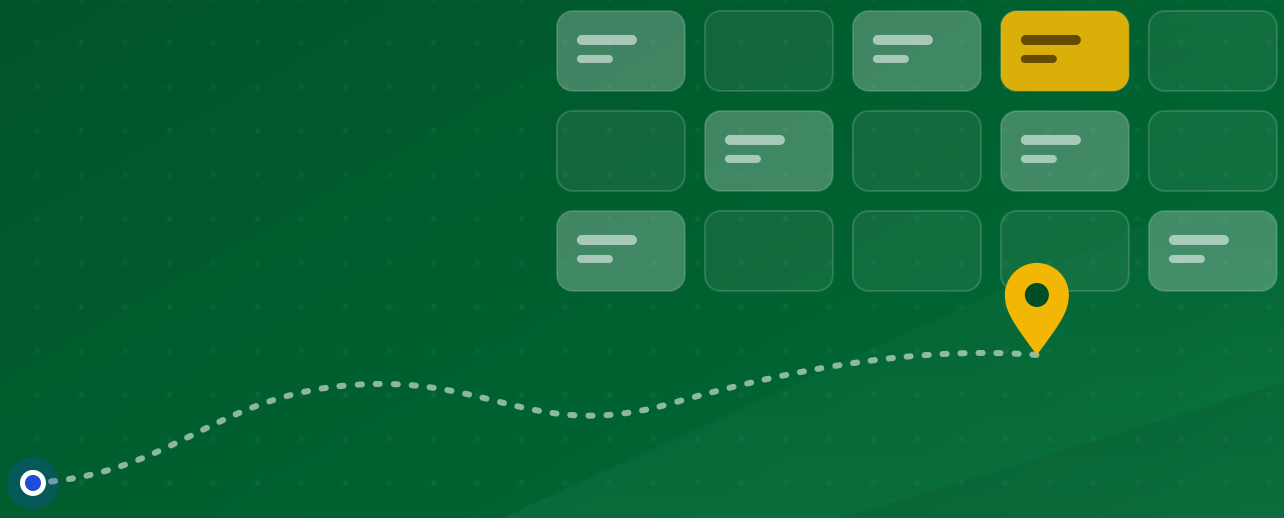
Yaba College of Technology
Labore et Veritate

PROJECT DOCUMENTATION

YCT Exam Portal

Smart Examination Timetable Generator & Real-Time Hall Navigation

Yaba College of Technology · School of Technology,
Department of Computer Science (pilot)



Project Documentation · October 2026
For college management, the exams office and academic staff

LIVE PILOT
`smart-exam-timetable.vercel.app`

PROJECT OVERVIEW

Contents

1 Executive summary 3

2 The challenge 4

3 The solution at a glance 5

4 Benefits 6

5 Who uses it 7

6 How it works 8

6.1 Architecture 8

6.2 Security and access control 9

6.3 Data model 10

6.4 The timetable engine 11

6.5 Timetable lifecycle 14

6.6 Hall navigation 15

7 The Exams Office portal 16

8 The Student portal 19

9 Running an exam period 21

10 The student journey 22

11 Technology 23

12 Deployment and operations 24

13 Looking ahead 25

A Appendix A: Codebase guide 26

B Appendix B: API reference 32

C Appendix C: Glossary 34

HOW TO READ THIS DOCUMENT

Sections 1 to 5 explain the purpose and value of the system for every reader. Section 6 explains how it works, with diagrams. Sections 7 to 10 tour the two portals and the day-to-day workflow. Sections 11, 12 and the appendices are reference material for technical staff, and section 13 looks ahead. Two companion handouts, the **Exams Office Guide** and the **Student Guide**, cover each audience step by step. Screens shown are illustrations with sample data.

1 Executive summary

What YCT Exam Portal is, why it matters and where it stands today.

YCT Exam Portal is a web application that produces Yaba College of Technology's examination timetable automatically, free of clashes, with every class placed in a hall that can hold it and every student given a numbered seat. It then guides each student to the right hall on the day, using the phone in their pocket.

The challenge

Every semester the timetable has been built by hand in a spreadsheet: checking which courses clash, which halls are free and whether each class will fit, then pinning seat lists on noticeboards. It takes days, mistakes are hard to spot, and students crowd round printed sheets before setting off to find a building some have never visited.

The solution

The exams office keeps the academic structure and exam halls in one system; students register and choose their courses themselves. One click creates the exam slots, schedules every course without clashes, books halls and numbers every seat. After review the officer publishes, and each student sees their own timetable, with live walking directions to the hall.

1 click
Full timetable
exam slots, halls and seats generated in seconds

0 clashes
By design
no level ever sits two papers at the same time

1 seat
For every enrolled student
numbered within the hall, in matric order

Live
Walking directions
from where the student stands to the exam hall

Who it serves

Exams office

Maintains schools, departments, courses and halls; generates, reviews and publishes the timetable; manages student accounts.

Students

Register with their matric number, choose their courses, see their personal timetable with hall and seat, and follow a walking route to the hall.

College management

Gains a transparent, repeatable process and one reliable source of truth for courses, halls and examinations.

WHERE IT STANDS

Piloted in the **School of Technology, Department of Computer Science** (shown in the app as "Computer Technology", code CSC) and live at `smart-exam-timetable.vercel.app`. The database is pre-loaded with Yabatech's **8 schools** and **47 departments** (each with ND and HND programmes and four levels, making **94 programmes and 188 levels**), about **1,000 course definitions**, two verified exam halls (Multipurpose Hall, code MPHA, 200 seats; Block C Lecture Hall, code BCLH, 80 seats), an optional set of 11 further campus buildings with 2,520 seats, a campus map graph and 50 demonstration students.

2 The challenge

How the examination timetable has been produced until now, and what that costs.

Every semester, the Department of Computer Science prepared its examination timetable by hand in a spreadsheet. Staff collected the course list for every level, checked by eye which papers could not sit together, found out which halls were free, estimated whether each class would fit, typed seat lists and pinned them on a noticeboard. The work took days and had to be partly repeated whenever something changed.

Days of work
Every timetable assembled and re-checked by hand, each semester.

Clashes slip through
Two papers for the same class can land in the same slot unnoticed.

Halls overflow
Without exact numbers, a class can be sent to a hall that is too small.

Crowds and lost students
Students queue at noticeboards, then search for unfamiliar buildings.

BEFORE: by hand, in a spreadsheet

1. Collect course lists for every level
2. Check for clashes by eye, level by level
3. Find out which halls are free, and when
4. Guess whether each class will fit its hall
5. Type seat lists for every paper
6. Print lists and pin them on noticeboards
7. Students crowd the board and hunt for halls

TYPICAL RESULT
Days of work every semester · clashes can slip through · halls can overflow · queues

AFTER: with YCT Exam Portal

1. **Structure, courses & halls in the portal** kept up to date by the exams office
2. **Students register and tick their courses** on their own phones, with their matric number
3. **One click: Generate** exam slots, a clash-free schedule, halls and numbered seats, in seconds
4. **Review the draft, then Publish All** department by department, adjusting if needed
5. **Each student sees exam, hall and seat** with a live walking route to the hall

RESULT
Seconds to generate · clash-free by design
hall capacity respected · personal timetables

Figure 1. Before and after. Seven manual steps that took days become a short, guided workflow in which the slow and error-prone parts are done by the system in seconds.

3 The solution at a glance

One shared system for the exams office and every student, on any phone or computer.

YCT Exam Portal is a web application: it runs in any modern browser, on phones and computers, with nothing to install. It replaces the whole manual process in five steps.

1. **The exams office keeps the college's structure in the system**
   Schools → departments → programmes (ND / HND) → levels (ND 1, ND 2, HND 1, HND 2) → courses, plus every exam hall with its seating capacity and GPS location.
2. **Students register themselves**
   With their matric number, department, programme and level. Then they tick the courses they are taking this semester.
3. **The officer clicks Generate**
   After choosing the first exam day and the length of the exam period, the system creates the time slots, places every course so that no level sits two papers at once, never schedules more exams in a slot than there are halls, books halls and gives every enrolled student a numbered seat. It then reports anything that needs attention.
4. **Review, then Publish**
   The officer checks the draft department by department, adjusts if needed and clicks Publish. Only then do students see it.
5. **Students open the portal on their phone**
   Their next exam with a countdown, date, time, hall and seat, and a **Navigate to Hall** button that shows a live walking route from where they stand.

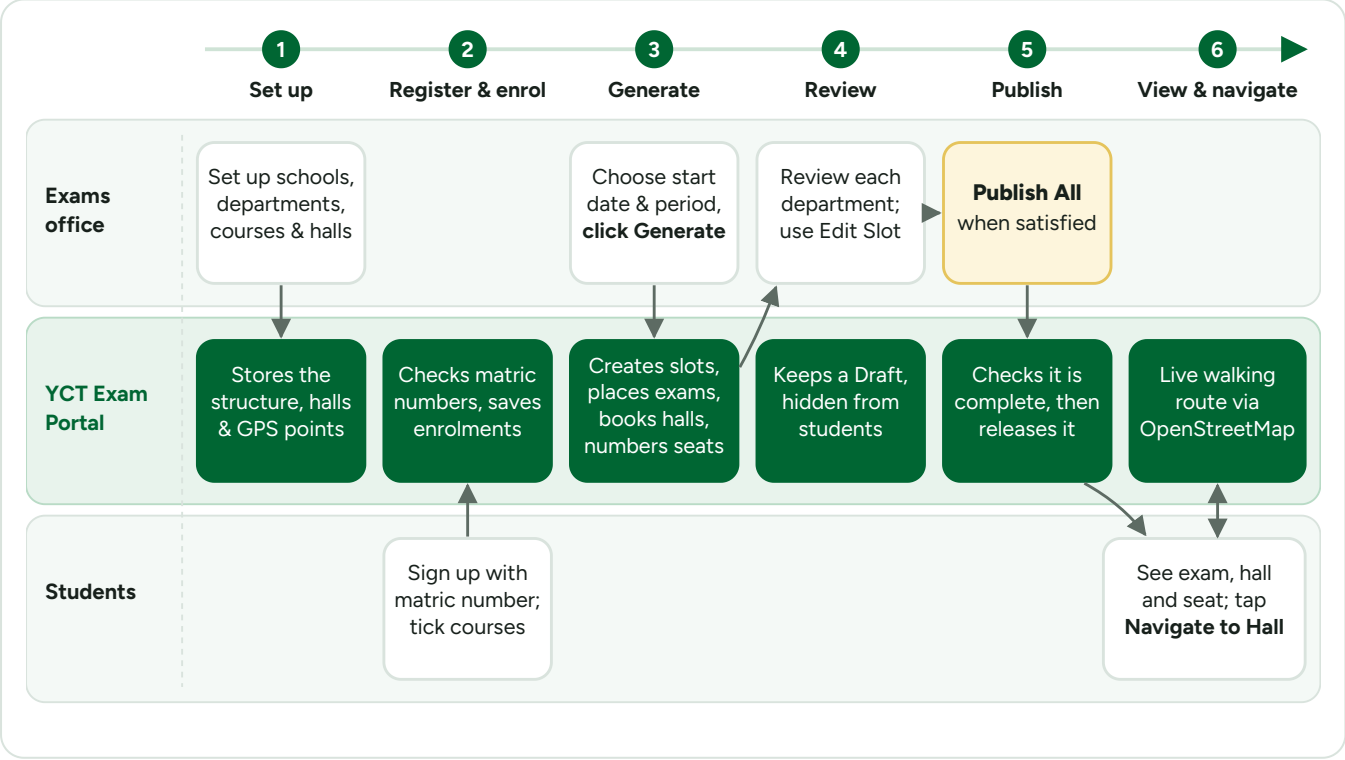


Figure 2. End-to-end flow. The exams office sets up and decides; the portal does the heavy lifting; students register, then view and navigate.

4 Benefits

What changes for the exams office, for students and for the college.

FOR THE Exams office

- **One click instead of days**
  The full timetable, halls and seats in seconds.
- **Re-run in seconds**
  After a change, generate again; the new draft replaces the old.
- **Clashes impossible by design**
  No level is ever given two papers at once.
- **Hall capacity always respected**
  No slot holds more exams than the halls can seat.
- **Consistent seat numbers**
  Produced automatically; the same input gives the same timetable.
- **Bulk CSV import**
  Departments and courses from a spreadsheet, with an import report.
- **Review, then publish**
  A clear draft-to-published workflow.
- **Complete student directory**
  Search, correct records, reset passwords.

FOR Students

- **A personal timetable**
  On their own phone, always up to date.
- **Exact hall and seat**
  No searching through printed lists.
- **Countdown to the next paper**
  "Today", "Tomorrow" or "In N days".
- **Walking directions**
  A live route from where they stand to the hall.
- **Own course registration**
  Enrol and drop courses themselves.
- **No crowding at noticeboards**
  Everything is on the phone.

FOR THE College

- **Fewer errors and complaints**
  Clashes and over-full halls are designed out.
- **Transparent and auditable**
  Draft first, published only when approved.
- **Data in one place**
  Courses, halls, students and timetables together.
- **Low running cost**
  Cloud hosting and open map data.
- **Modern, widely used technology**
  Easy to maintain and to find skills for.
- **Ready to scale**
  From the pilot department to more departments and schools.

Seconds
To generate or re-generate
instead of days of spreadsheet work

Repeatable
Same input, same timetable
results can be checked and reproduced

Any device
Phone or computer
nothing to install, light and dark themes

5 Who uses it

Three roles, two portals and one sign-in page.

Role	Portal and pages	What they can do	How the account is created
Student	Student portal: `/dashboard` (Timetable), `/courses`, `/navigate`, `/profile`	Register; choose courses; view the published timetable with hall and seat; get walking directions to any exam hall; manage their profile, avatar and theme.	Self-registration at `/register`
Exams officer (admin)	Exams Office portal under `/admin/…`	Manage schools, departments, courses and halls; create and activate sessions; generate, review, adjust and publish the timetable; view seat assignments; manage student accounts.	Created by the system administrator
Super admin	Exams Office portal under `/admin/…`	The same permissions as an exams officer.	The first account, created during setup

One sign-in page for everyone

Everybody signs in at `/login`. The system recognises each person's role and sends them to the right portal; if they originally asked for a particular page, they are taken back to it after signing in. Admin accounts are never created by public sign-up.

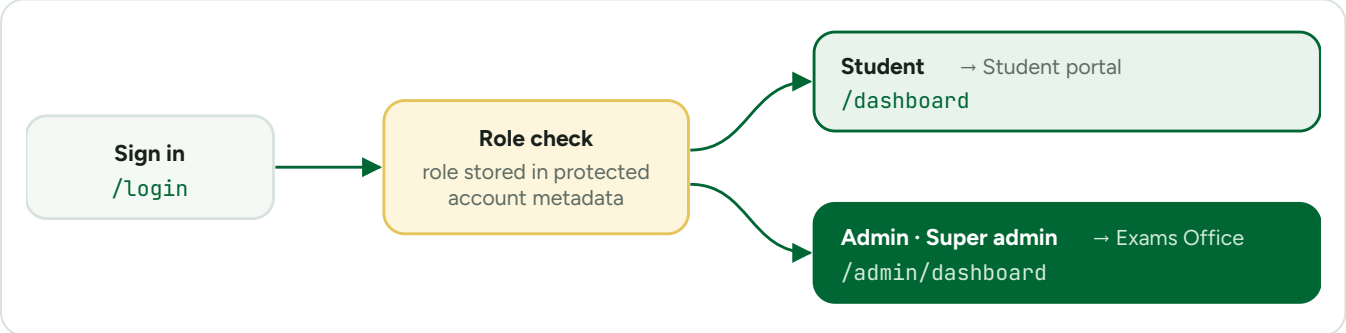


Figure 3. Sign-in routing. One page, two destinations; after signing in, people return to the page they first asked for.

GOOD TO KNOW

Exams officers and the super admin have the same permissions in the app. The super admin is simply the first account created when the system is set up; further admin accounts are added by the system administrator.

6 How it works

Architecture, security, data, the timetable engine and navigation, explained for a mixed audience.

6.1 Architecture

Everything a user does starts in the browser and passes through an access gateway before it reaches the application. The application's pages and its REST API share one codebase; the scheduling engine is a set of focused services; and all data lives in a managed PostgreSQL database on Supabase.

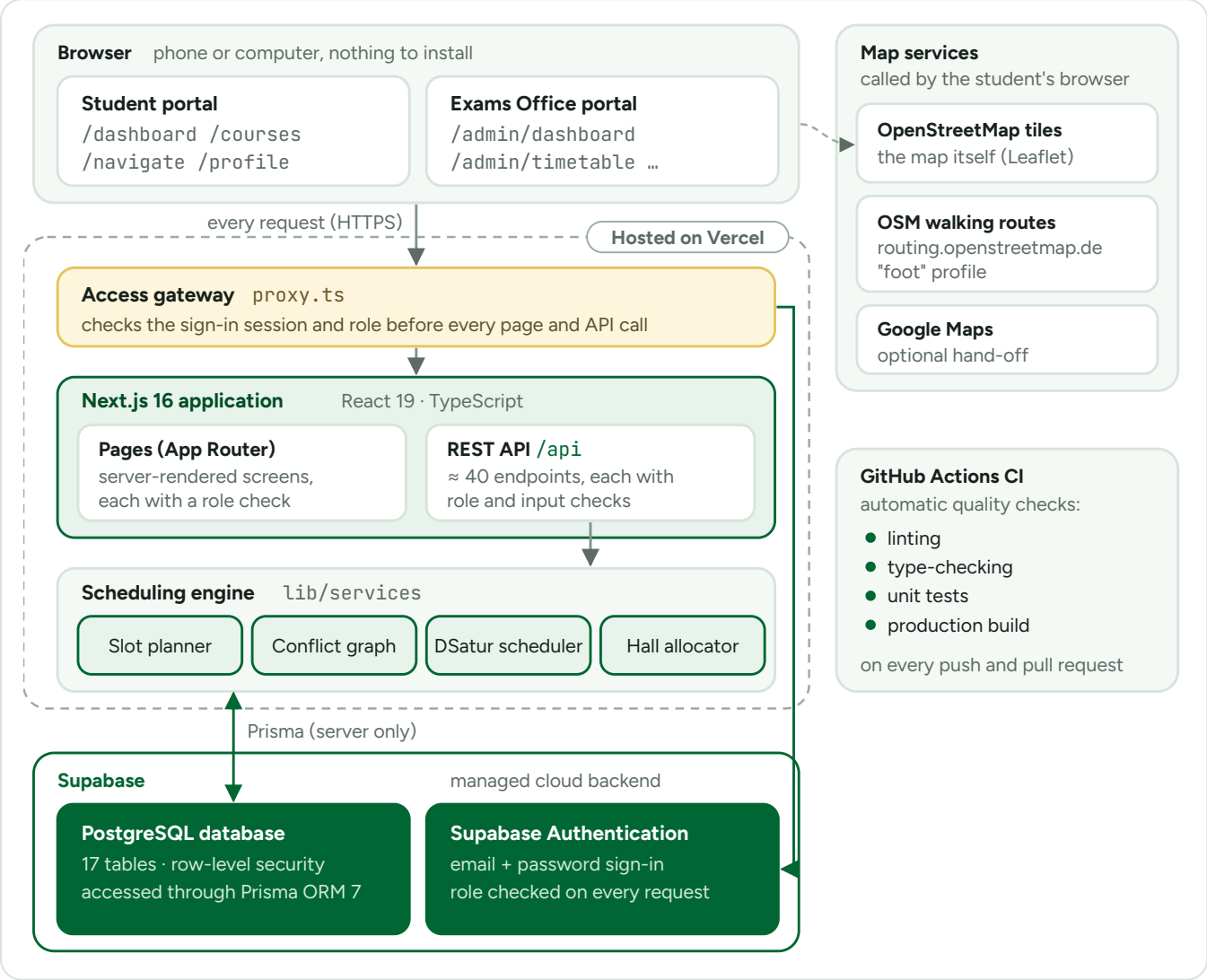


Figure 4. Architecture. The access gateway checks every request; the application and engine run on Vercel; Supabase provides the database and sign-in; map services are called directly by the student's browser.

Layer	What it does	Built with
Browser	Two portals in one web app: the Student portal (designed phone-first) and the Exams Office portal.	React 19, Tailwind CSS 4, shadcn/ui
Access gateway	Runs before every request: refreshes the sign-in session, checks the role, redirects or refuses.	`proxy.ts` (Next.js 16)
Application	Server-rendered pages and a REST API of about 40 endpoints under `/api`.	Next.js 16 App Router, TypeScript
Scheduling engine	Plans exam slots, builds the conflict graph, schedules with DSatur, allocates halls and seats.	`lib/services`
Data and sign-in	17 tables in PostgreSQL with row-level security; email and password sign-in with roles.	Supabase, Prisma ORM 7
Maps	Map tiles, walking routes and an optional hand-off to Google Maps.	Leaflet, OpenStreetMap
Hosting and quality	Hosted on Vercel; GitHub Actions lints, type-checks, tests and builds every change.	Vercel, GitHub Actions

6.2 Security and access control

Access is checked at three independent points on every request, so each page and each action is only ever available to the people it is meant for.

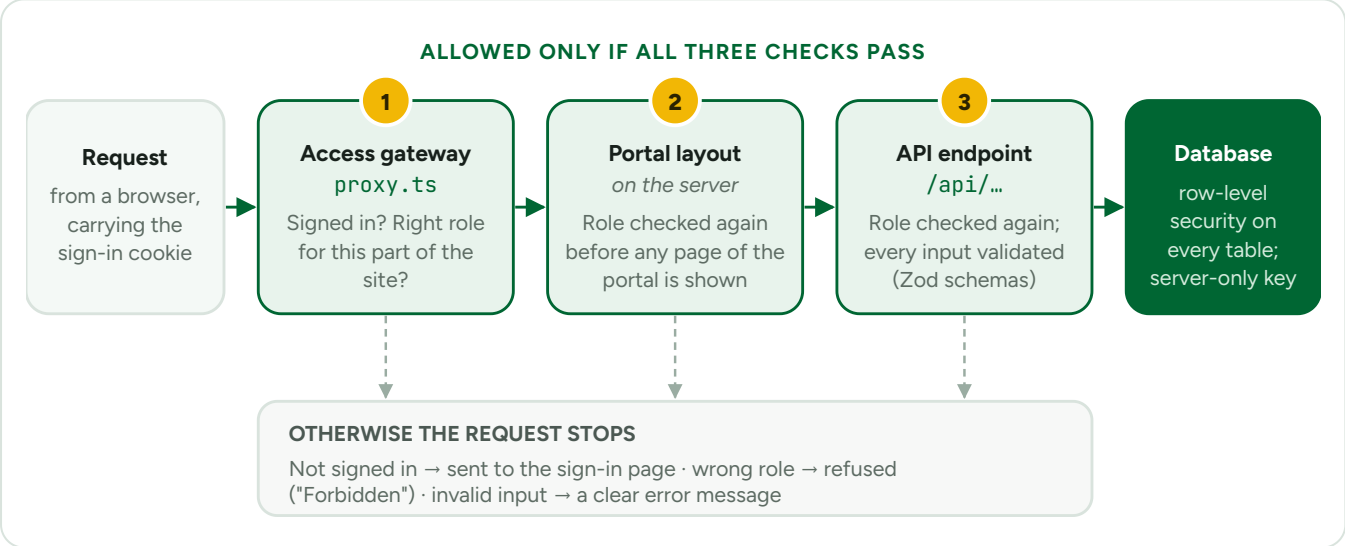


Figure 5. Three checkpoints. The gateway, the portal's server-side layout and every API endpoint each verify the role; the database itself is closed to direct access from browsers.

Roles cannot be self-assigned

Roles live in protected account metadata that only the server can write, so nobody can promote themselves.

Students only see their own data

Students never send their own ID; the server identifies them from their sign-in, so they can only ever read and change their own records.

Passwords handled by Supabase

Sign-in is provided by Supabase Authentication; the application never stores passwords.

Row-level security on every table

The public key used by browsers cannot read the tables; only the server, with its private credentials, talks to the database.

Every input validated

Forms and API requests are checked with the same Zod rules in the browser and on the server.

Clean, unique matric numbers

Matric numbers are normalised (`f/nd/24/3210001` → `F/ND/24/3210001`) and checked for duplicates regardless of letter case.

Admins can correct student records, reset a student's password or remove an account from the Students screen.

6.3 Data model

Seventeen tables hold the academic structure, the people, course registration, examinations, the campus map and one operational safeguard.

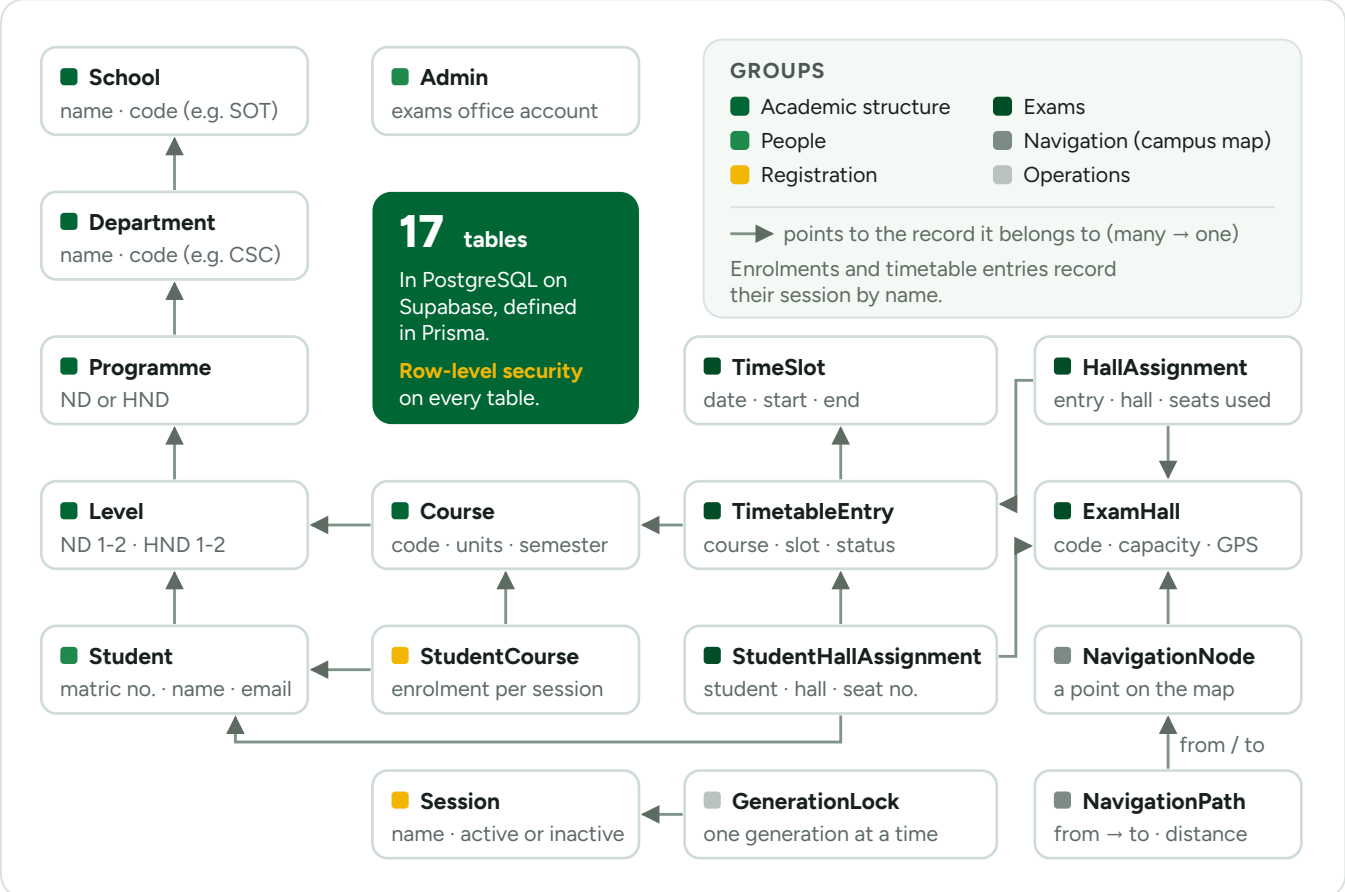


Figure 6. Data model. The academic tree runs School → Department → Programme → Level → Course. Enrolments link students to courses for a session; timetable entries place courses in time slots; hall assignments and seat assignments record where each exam and each student sits.

Group	Tables and what they hold
Academic structure	**School**, **Department**, **Programme** (ND / HND), **Level** (ND 1, ND 2, HND 1, HND 2) and **Course** (code, title, credit units 1-6, First or Second semester).
People	**Student** (matric number, name, email, level, programme and department, linked to a sign-in account) and **Admin**.
Registration	**StudentCourse** is an enrolment: student + course + session, e.g. "2025/2026 First Semester". **Session** is the list of academic sessions; each can be Active (visible to students) or Inactive.
Exams	**TimeSlot** (date, start and end), **TimetableEntry** (course + slot + session, Draft or Published), **HallAssignment** (which halls an exam uses and how many seats), **StudentHallAssignment** (each student's hall and seat number), **ExamHall** (name, code, capacity, latitude/longitude, active).
Navigation	**NavigationNode** and **NavigationPath**: the campus map graph of points and walkable links.
Operations	**GenerationLock** makes sure only one timetable generation runs for a session at a time.

6.4 The timetable engine

When the officer clicks **Generate**, the engine turns courses, enrolments and halls into a complete draft timetable in nine steps. The officer chooses three inputs in the Generate dialog: the **School** ("All Schools" or one school), the **Exam Start Date** (next Monday by default) and the **Exam Period** (1 to 6 weeks, two weeks by default).

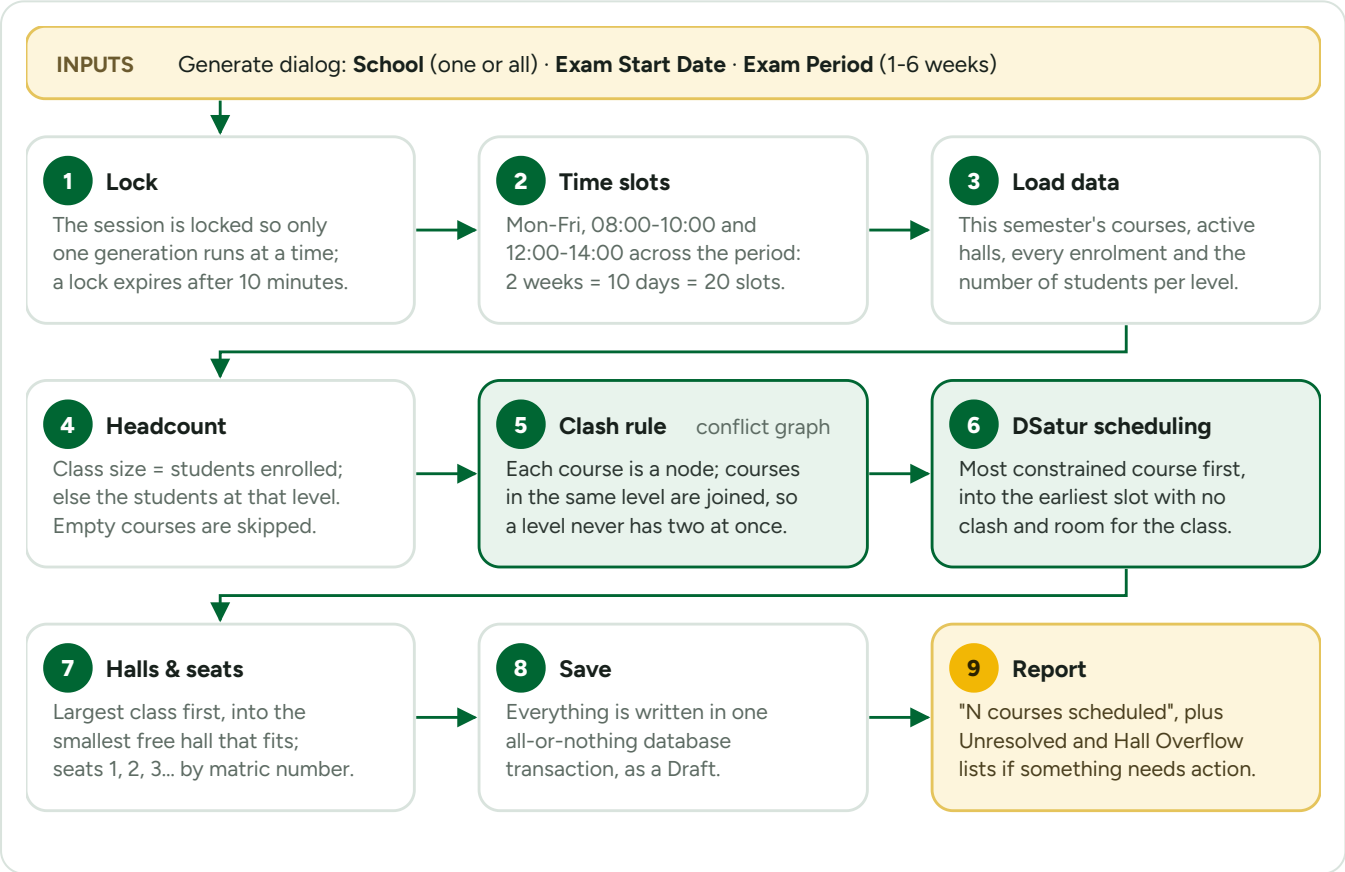


Figure 7. The nine-step pipeline. Steps 5 and 6 are the heart of the engine: a conflict graph and the DSatur graph-colouring algorithm, in which every time slot is a "colour".

#	Step	What happens
1	**Lock**	The session is locked so two generations cannot run at the same time. A lock left by an interrupted run expires automatically after 10 minutes.
2	**Time slots**	Monday to Friday, two papers a day (**Morning 08:00-10:00** and **Afternoon 12:00-14:00**) for the whole exam period (2 weeks = 10 weekdays = 20 slots). Only slots inside the exam period are used.
3	**Load data**	The courses for the semester (the session name says First or Second), the active halls, every student's enrolments and the number of students at each level.
4	**Headcount**	Class size is the number of students enrolled in the course; if nobody has enrolled yet, the number of students registered at that level. Courses with no students are skipped.
5	**Clash rule**	Every course is a node; two courses in the same level are joined by an edge, because the same students sit both. No level ever has two papers at once. This protects every student, including those still finishing enrolment.
6	**DSatur scheduling**	The engine repeatedly takes the most constrained course, the one whose clashing courses already occupy the most different slots (ties decided by number of clashes, then class size, then a fixed order), and places it in the **earliest slot that holds no clashing course and still has enough free hall space for the whole class**. Halls already booked in that slot by another timetable (another school or session) are treated as unavailable.
7	**Halls and seats**	Within each slot the largest class chooses first and takes the **smallest free hall that fits the whole class**; a class too big for any single hall fills the largest free hall and continues in the next. A hall holds one course per slot. Seats are numbered 1, 2, 3… inside each hall, in matric-number order.
8	**Save**	Everything is written in one all-or-nothing database transaction, as a **Draft**.
9	**Report**	"N courses scheduled", plus **Unresolved** (no clash-free slot left, so choose a longer exam period) and **Hall Overflow** (not enough hall space, so add halls or capacity) when needed. Generating again replaces the draft.

DSATUR IN PLAIN ENGLISH

Think of each time slot as a colour and each course as a dot; dots joined by a line must get different colours. DSatur ("degree of saturation") always colours next the dot that is hardest to place, the one already surrounded by the most colours, so difficult courses are placed while there is still room. Because ties are always decided in the same fixed order, the same input always produces the same timetable.

Worked example

Three levels (ND 1 with four courses, ND 2 with three and HND 1 with three) and two halls. The conflict graph on the left joins courses of the same level. On the right, each column is a time slot: no column ever holds two courses of the same level, and no column holds more than two exams, because there are only two halls.

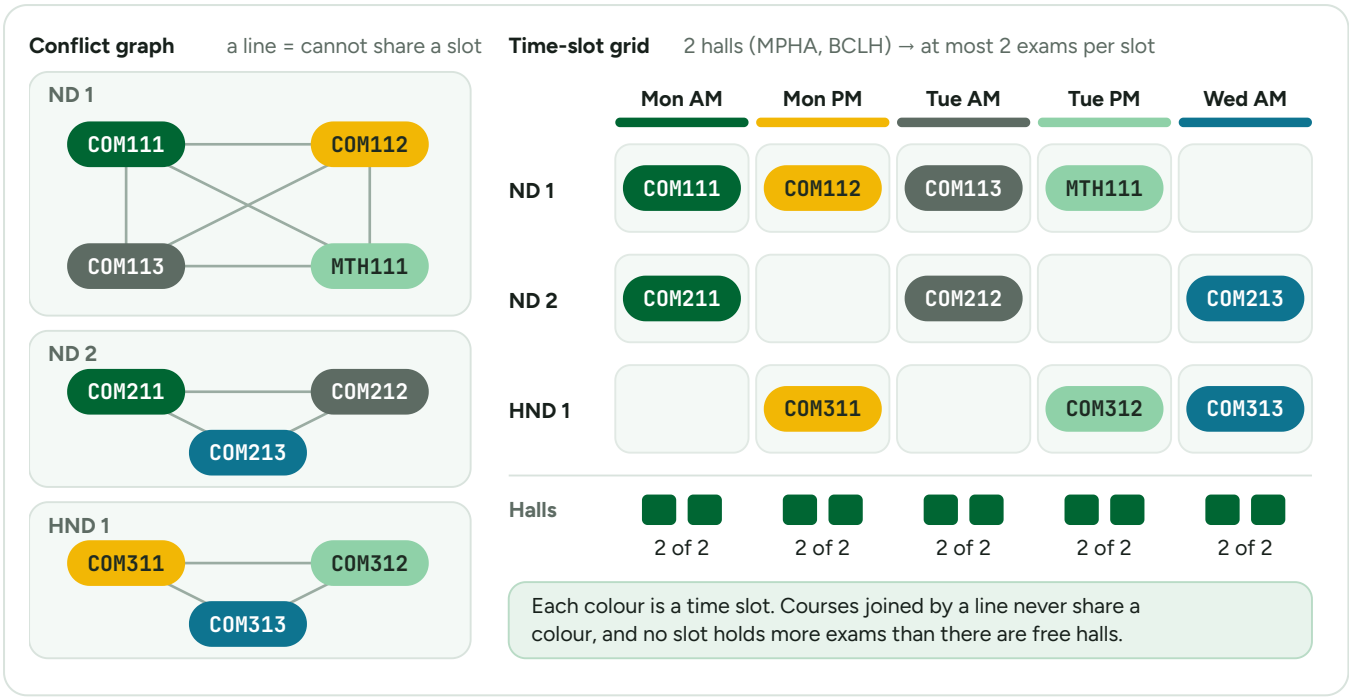


Figure 8. Worked example (illustrative): one clash-free arrangement of ten exams for three levels in five slots with two halls. Each course carries the colour of its slot.

Halls and seats

Once a course has its slot, it is given halls and every enrolled student receives a seat. Best fit keeps big halls free for big classes; a class larger than any hall is split across halls automatically.

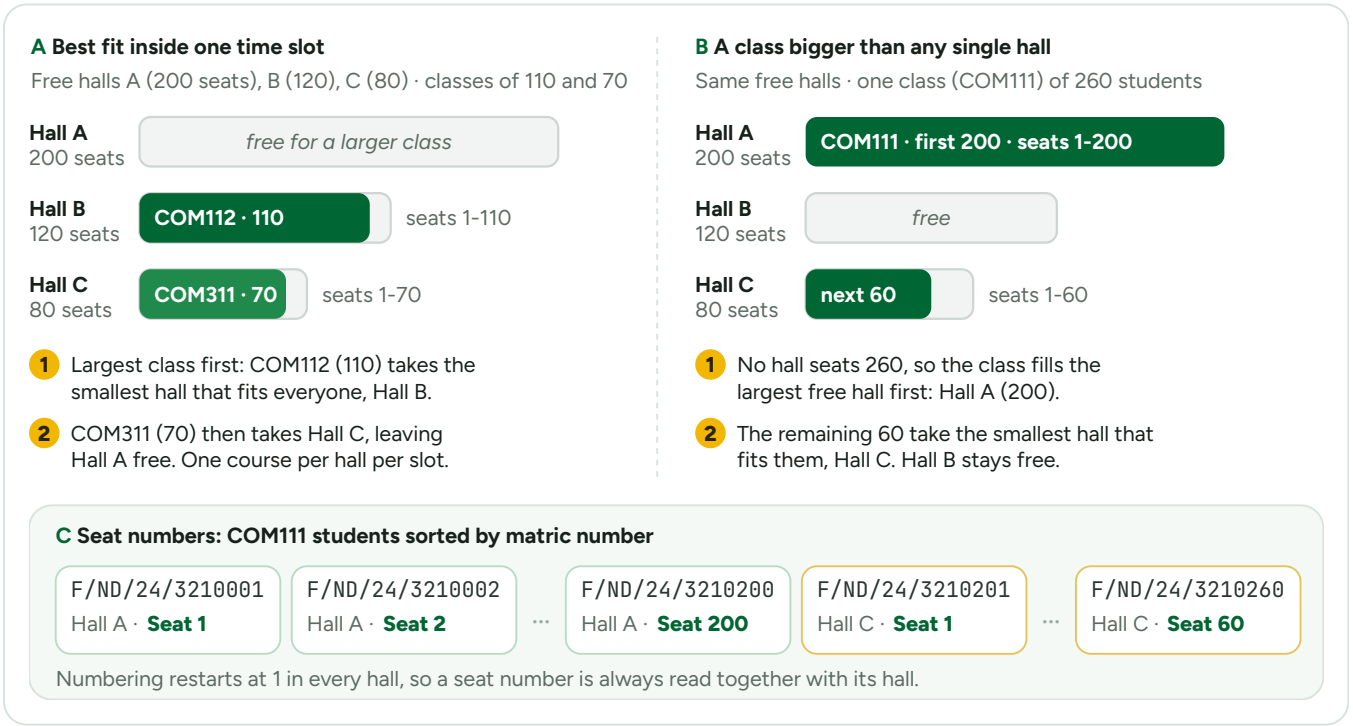


Figure 9. Best fit and seat numbering (sample halls). Larger classes choose first and take the smallest hall that fits; oversized classes fill the largest free hall and continue in the next; seats are numbered from 1 in each hall, in matric-number order.

6.5 Timetable lifecycle

A timetable is always created as a private **Draft** and becomes visible to students only when the officer publishes it.

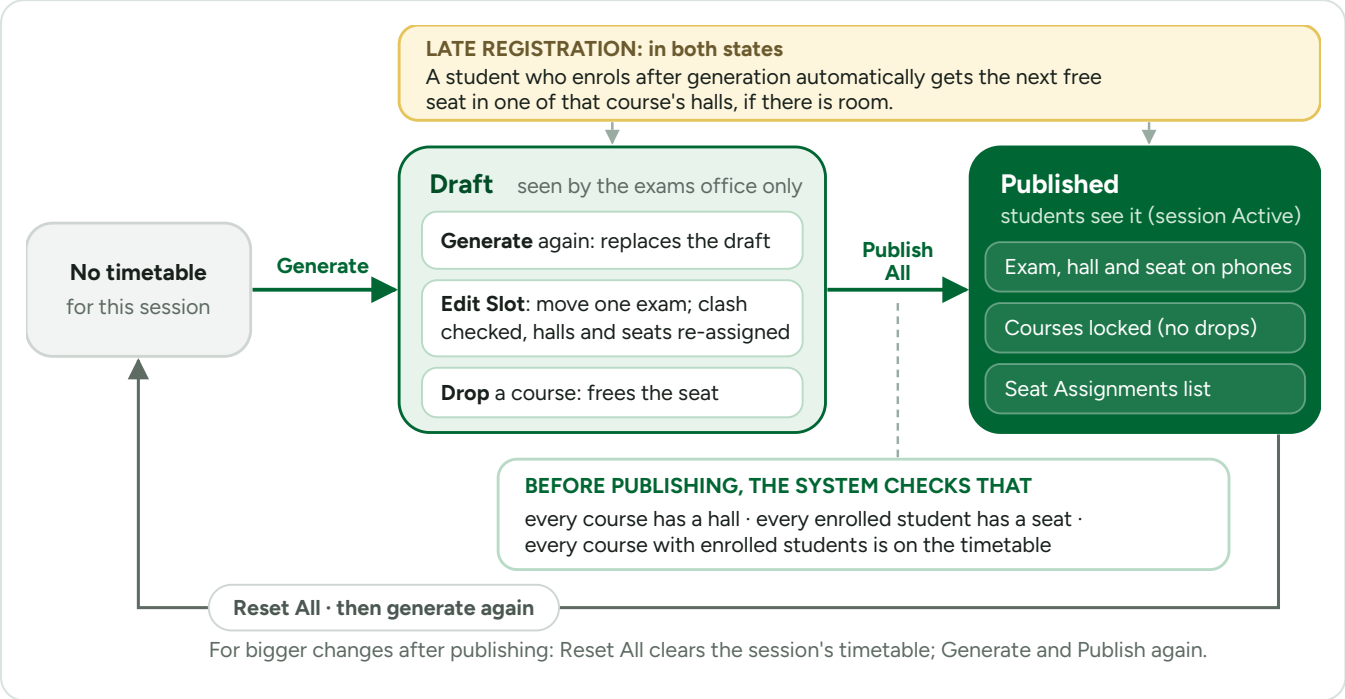


Figure 10. Lifecycle. Generate creates a Draft that can be regenerated or adjusted with Edit Slot; Publish All releases it after completeness checks; Reset All starts again for larger changes.

- **Edit Slot** (drafts only) moves one exam to another slot. The system checks for clashes and automatically re-assigns halls and seats in the new slot.
- **Publish All** releases the timetable to students after checking that every scheduled course has a hall, every enrolled student has a seat, and every course with enrolled students is on the timetable.
- **Late registration:** a student who enrols after the timetable has been generated is automatically given the next free seat in one of that course's halls, if there is room. Dropping a course before publication frees the seat, and a dropped course can be added again.
- **After publishing**, students can no longer drop published courses. For bigger changes the officer uses **Reset All**, then generates and publishes again.
- **Seat Assignments** lists every student's course, date, hall and seat, ready to share with invigilators.

6.6 Hall navigation

The Navigate tab lists every active exam hall with a small map preview, its code, description and coordinates. Opening a hall, or tapping **Navigate to Hall** on the timetable, shows a full-screen map with the hall (green pin), the student's live position from the phone's GPS (blue dot) and the walking route, with distance and time such as "850 m · 11 min walk".

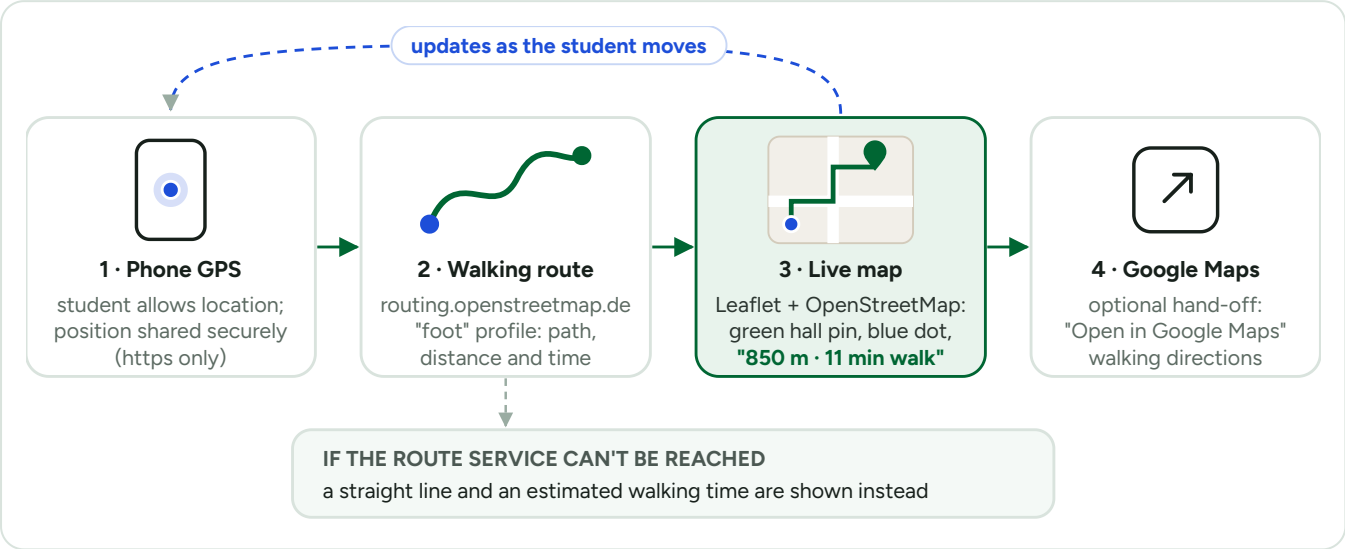


Figure 11. Hall navigation. The phone's position and OpenStreetMap's walking-route service produce a live route on the map; Google Maps is one tap away.

- The route **updates as the student moves**. If the route service cannot be reached, a straight line and an estimated walking time are shown instead.
- **Open in Google Maps** hands off to Google Maps walking directions for turn-by-turn guidance.
- Location is shared only with the student's permission, and browsers provide it over a secure (https) connection.
- A campus map graph (navigation nodes and paths) with a Dijkstra shortest-path API, `/api/navigation/path`, is also available for campus-specific routing.

7 The Exams Office portal

A guided tour of the screens the exams office uses every semester.

The portal opens on the **Dashboard**. A sidebar lists **Dashboard, Schools, Departments, Courses, Halls, Timetable** and **Students**; the header shows the page name and a light/dark switch; the sidebar footer shows the signed-in email, role and **Sign out**. **Ctrl/Cmd + B** collapses the sidebar. Every list is a sortable table on computers and a set of cards on phones; deletions always ask for confirmation, and every action confirms success or explains an error.

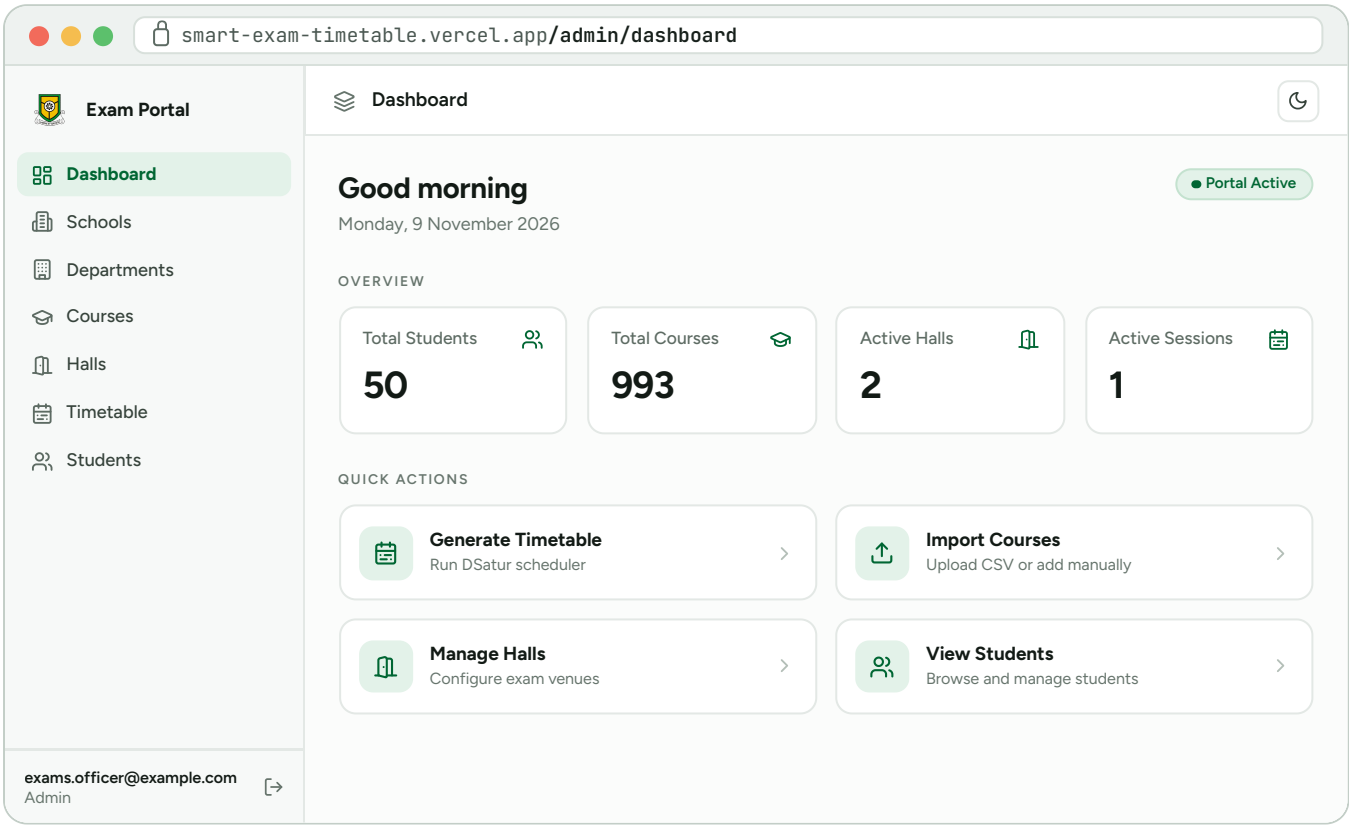


Figure 12. Dashboard: greeting, today's date, four headline figures and four quick actions. ILLUSTRATION (SAMPLE DATA)

Screen	Purpose and main actions
Dashboard	Stat cards: *Total Students, Total Courses, Active Halls, Active Sessions*. Quick actions: *Generate Timetable, Import Courses, Manage Halls, View Students*.
Schools	Add, edit and delete schools (name and code).
Departments	Filter by school. *Add Department* also creates its two programmes, "National Diploma (ND)" and "Higher National Diploma (HND)", and the four levels ND 1, ND 2, HND 1, HND 2. *Upload CSV* (`school_name,name,code`) with an import report of imported, skipped and rejected rows.
Courses	Cascading filters School → Department → Programme → Level; columns Code, Title, Level, Credits, Semester; pages of 50. *Add Course* and *Upload CSV* (`course_code,course_title,department_code,level_name,credit_units,semester`; existing codes are updated).
Halls	Name, Code, Capacity, Coordinates, Status. *Add Hall* with latitude, longitude, description and Active. Inactive halls are not used for new timetables and are hidden from students.
Timetable	The command centre: sessions, Generate, the result report, department tabs, Edit Slot, Publish All, Seat Assignments and Reset All.

Screen	Purpose and main actions
Students	Search by name or matric number; filter by school, department and level; 50 per page. *View* opens a profile with enrolments; actions *Edit*, *Reset password*, *Delete*.
Advanced	Opened by URL when needed: `/admin/programmes`, `/admin/levels` and `/admin/slots`.

The Timetable screen: the command centre

Choose a session, or create one with **+** ("YYYY/YYYY First Semester" or "Second Semester"). The **Active / Inactive** switch controls whether students can see the session; a session can also be deleted. The toolbar offers **Generate**, **Publish All**, **Seat Assignments** and **Reset All**; department tabs split the timetable for review, with columns Code, Title, Level, Date, Time, Halls and Status (Draft / Published) and an **Edit Slot** action on drafts.

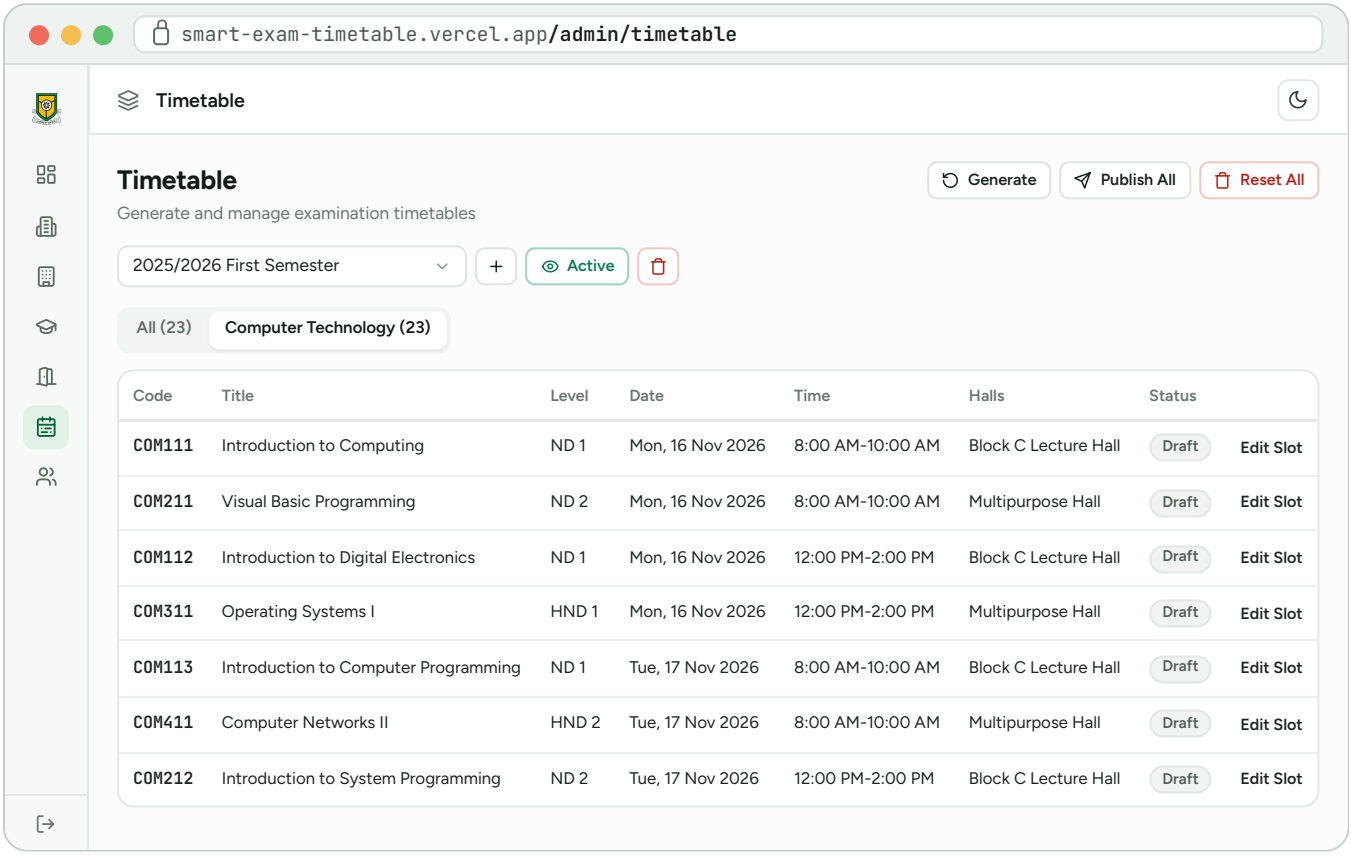
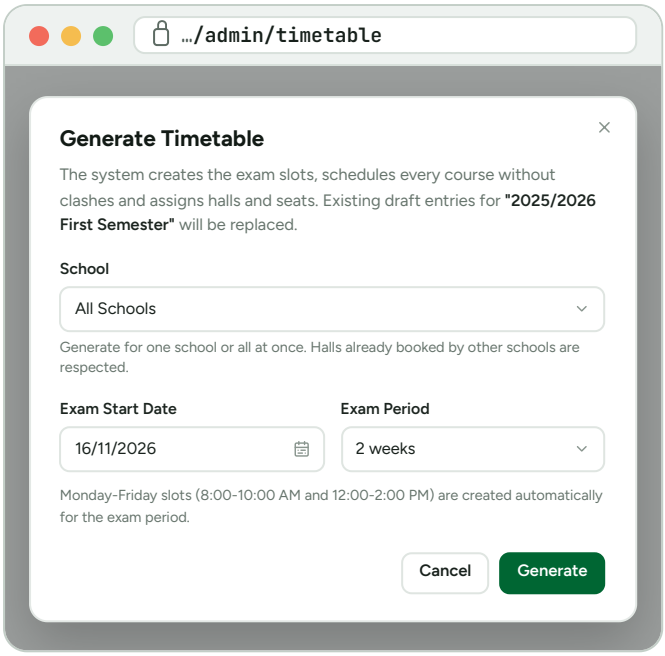
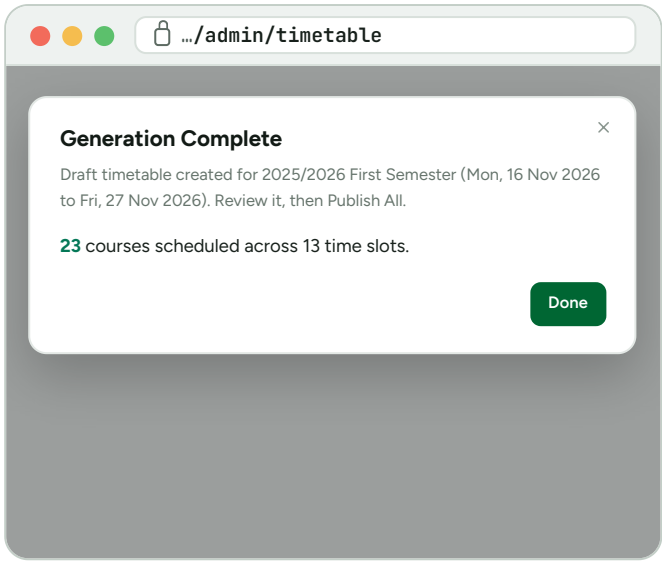


Figure 13. Timetable: a draft for the Computer Technology department, ready for review (sidebar collapsed with Ctrl/Cmd + B). ILLUSTRATION (SAMPLE DATA)



Generate: choose the school, first exam day and length of the exam period.

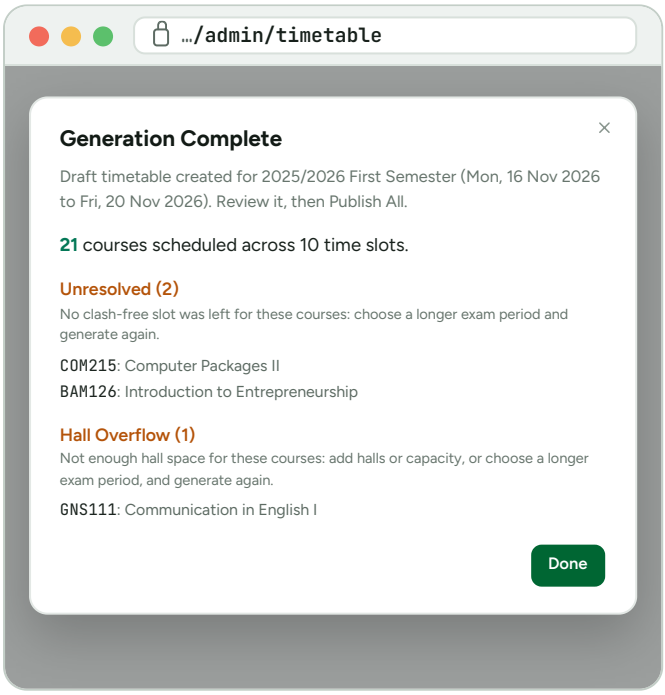


Generation Complete: the report. *Unresolved* and *Hall Overflow* lists are added only when something needs attention.

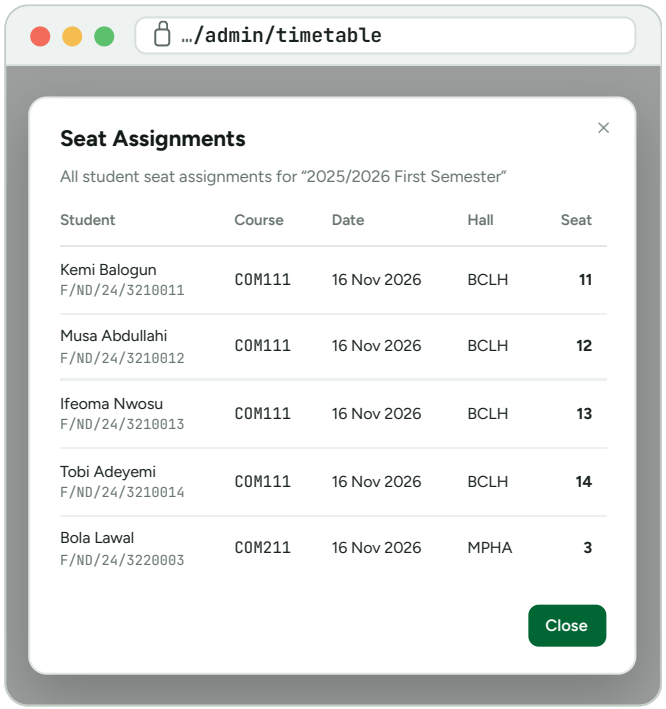
Figure 14. Generate and the result report.

ILLUSTRATION (SAMPLE DATA)

When a run needs attention, the report names the courses concerned and the remedy: a longer exam period for **Unresolved** courses, more hall space for **Hall Overflow**. Once the timetable is published, **Seat Assignments** lists every student's course, date, hall and seat for invigilators.



When something needs attention, each list names the courses concerned and what to do.



Seat Assignments: every student's course, date, hall and seat, for invigilators.

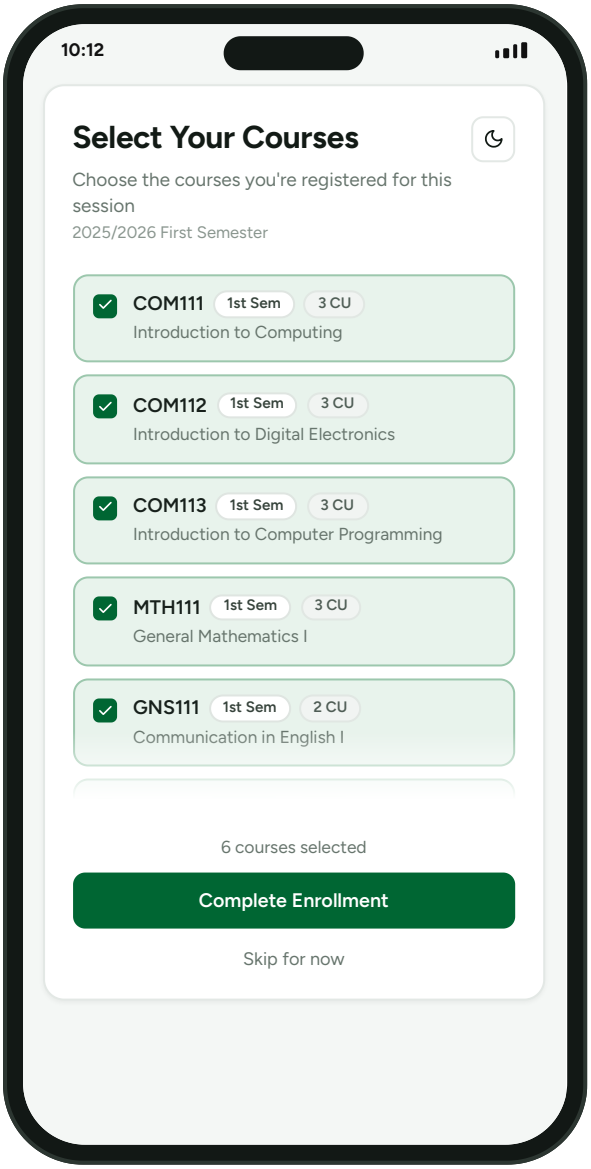
Figure 15. A report with items to act on, and the Seat Assignments list.

ILLUSTRATION (SAMPLE DATA)

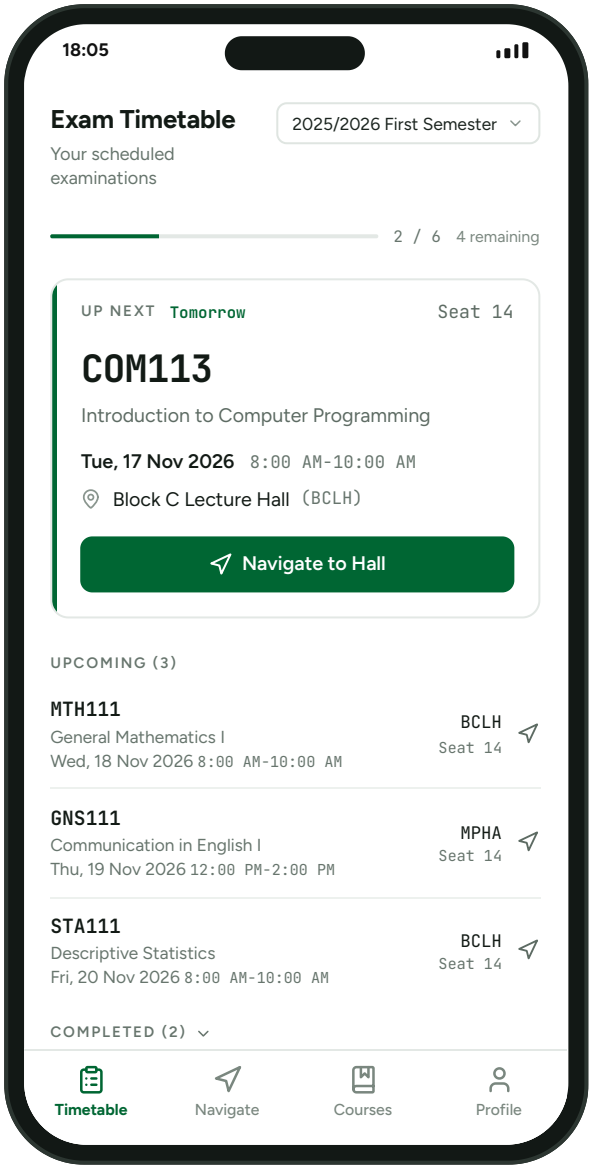
8 The Student portal

Designed for the phone in every student's pocket, and just as good on a computer.

Students sign up at `/register` with their Full Name, Matric Number, Email, Department → Programme → Level and a password, and are signed in immediately. They then see **Select Your Courses**: the courses for their level and the current semester, all pre-ticked. They untick any electives they are not taking and tap **Complete Enrollment** (or *Skip for now*). On a computer the portal has a top bar (Timetable, Navigate, Courses, Profile, theme switch, Sign out); on a phone the same tabs sit in a bottom tab bar.



Select Your Courses: right after sign-up, everything pre-ticked.



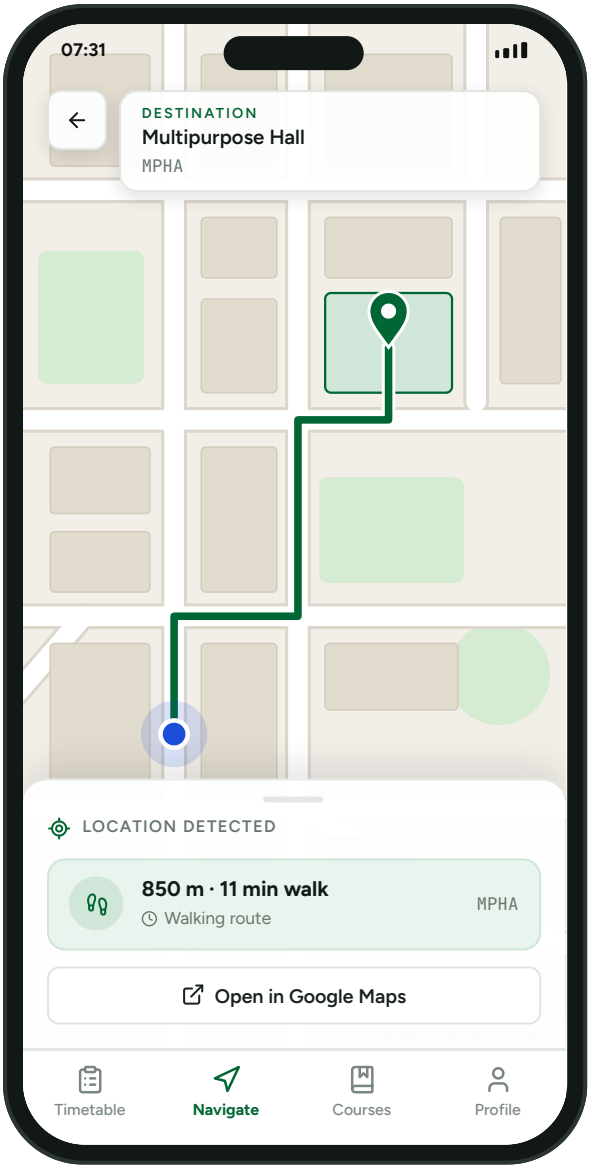
Timetable: Up Next with countdown, hall and seat.

Figure 16. Course selection and the personal timetable. ILLUSTRATION (SAMPLE DATA)

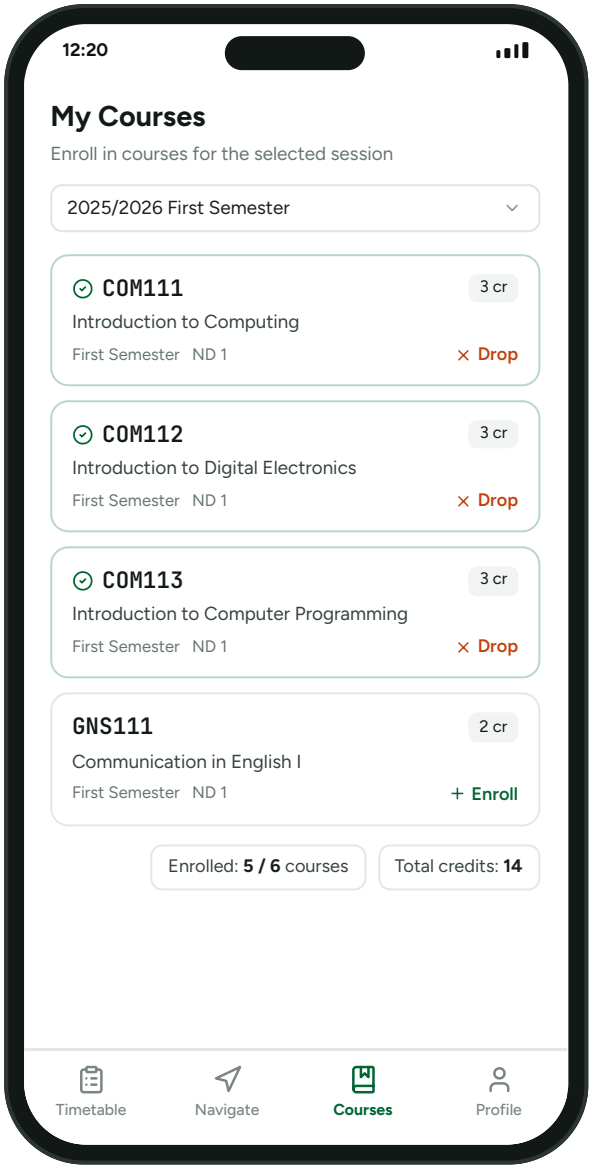
Timetable

The Timetable tab has a session selector and a progress bar ("2 / 6", "4 remaining"). The **Up Next** card shows a countdown (Today, Tomorrow or In N days), the seat number, course code and title, date, time, and the hall's name and code, with a **Navigate to Hall** button. Below are the **Upcoming** exams and a collapsible **Completed** list

(an exam moves there once it has finished), ending with "All exams completed". Only published timetables appear. If a hall is shown before a seat number has been assigned, the student sees "Seat number not yet assigned: please contact the exams office".



Navigate: live walking route, distance and time.



Courses: enrol and drop; locked once published.

Figure 17. Directions to the hall and course management. ILLUSTRATION (SAMPLE DATA)

Navigate

A directory of every active hall, and a full-screen map for each: the hall, the student's position and the walking route, with **Open in Google Maps** for turn-by-turn directions.

Courses and Profile

Courses lists the level's courses for the selected session's semester with *Enroll* and *Drop* (with confirmation); published courses show *Locked*, alongside the enrolled count and total credit units. The chosen session is remembered between Timetable and Courses. **Profile** shows the student's details, an emoji avatar, the light/dark switch and Sign out.

9 Running an exam period

The exams officer's checklist, from set-up to the last paper.

1. **Check the academic structure**
   Schools, departments and courses are in place. Adding a department creates its ND and HND programmes and four levels automatically; courses can be imported in bulk from CSV.
2. **Confirm the halls**
   Every hall in use has its real seating capacity and GPS coordinates and is marked Active. Halls no longer used are set to Inactive.
3. **Prepare the session**
   On the Timetable screen, create the session with **+** (e.g. "2025/2026 First Semester") and make it **Active**, so students can see it.
4. **Open student registration**
   Students sign up at `/register` and choose their courses. The Students screen shows who has registered.
5. **Generate**
   When registration closes, click **Generate** and choose the School ("All Schools" or one), the Exam Start Date and the Exam Period.
6. **Read the report**
   "N courses scheduled". If **Unresolved** courses are listed, choose a longer exam period; if **Hall Overflow** is listed, add halls or capacity. Then generate again.
7. **Review department by department**
   Use the department tabs to check dates, times and halls. **Edit Slot** moves a single exam; clashes are checked and halls and seats re-assigned automatically.
8. **Publish All**
   The system confirms that every course has a hall, every enrolled student has a seat and every course with students is scheduled. Students can then see their timetable.
9. **Share Seat Assignments**
   The list of every student's course, date, hall and seat is ready for invigilators.
10. **During the exam period**
    Late registrations receive the next free seat automatically when there is room. For bigger changes, use **Reset All**, then generate and publish again.

TIP

Keep only the current session Active, generate after registration has closed, and look through every department tab before publishing. The Exams Office Guide covers each step in detail.

10 The student journey

From creating an account to finding the right seat in the right hall.

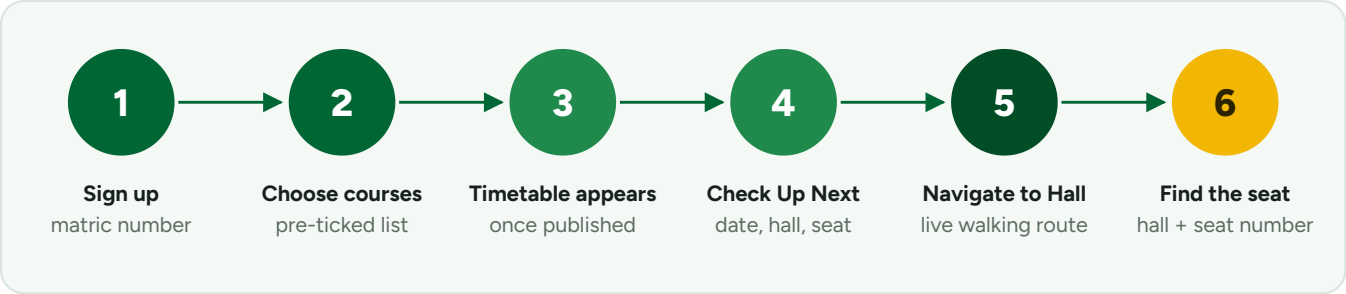


Figure 18. The student journey at a glance.

1. **Create an account**
   At `/register`: Full Name, Matric Number, Email, Department → Programme → Level, Password and Confirm Password, then *Create account*. The student is signed in straight away.
2. **Select Your Courses**
   The courses for their level and the current semester appear, all ticked. They untick electives they are not taking and tap *Complete Enrollment*.
3. **Wait for publication**
   The timetable appears on the Timetable tab as soon as the exams office publishes it, so there is nothing to check on a noticeboard.
4. **Check what is next**
   The Up Next card shows the countdown, date, time, hall and seat; Upcoming lists the rest; finished papers move to Completed.
5. **Navigate to Hall**
   On exam day, tap the button, allow location access, and follow the live walking route, or open it in Google Maps.
6. **Take the right seat**
   Seat numbers start at 1 in each hall, so the student reads hall and seat together.
7. **Keep courses up to date**
   On the Courses tab students enrol in or drop courses. Once a course's timetable is published it shows *Locked* and stays on their list; a late enrolment still receives a seat when there is room.

HELP WHEN IT IS NEEDED

Forgotten passwords and corrections to a student's department or level are handled by the exams office, which can update records and reset passwords. The Student Guide answers the most common questions.

11 Technology

A modern, widely used and well-supported stack, chosen for reliability, low cost and ease of maintenance.

Area	Technology	Why this choice
Web framework	Next.js 16 (App Router) + React 19	Pages and API in one codebase; fast server-rendered screens; first-class hosting on Vercel.
Language	TypeScript (strict mode)	Catches mistakes before they reach users and makes the code self-documenting.
Database	PostgreSQL on Supabase	A proven relational database, fully managed, with row-level security.
Database access	Prisma ORM 7	Type-safe queries, versioned migrations and one schema file as the source of truth.
Sign-in	Supabase Authentication	Email and password sign-in with roles; passwords are never stored by the application.
Data fetching	TanStack Query	Caching and automatic refresh keep screens fast and up to date.
Admin tables	TanStack Table	Sortable, paginated tables for long lists of courses and students.
Forms and validation	React Hook Form + Zod	The same validation rules run in the browser and on the server.
Interface	Tailwind CSS 4 + shadcn/ui on Base UI	Consistent, accessible components with light and dark themes.
Maps	Leaflet + OpenStreetMap	Open map data with no licence fees.
Walking routes	OpenStreetMap routing service ("foot" profile)	Real walking paths with distance and time.
Directions hand-off	Google Maps	Familiar turn-by-turn directions, one tap away.
Automated tests	Node's built-in test runner via `tsx` (`npm test`)	Fast unit tests of the scheduling rules, with no extra test framework.
Continuous integration	GitHub Actions	Linting, type-checking, tests and a production build on every change.
Hosting	Vercel	Automatic HTTPS, global delivery and no servers to maintain.

OPEN STANDARDS, LOW RUNNING COST

The map data comes from OpenStreetMap, the database is standard PostgreSQL, and the whole application is written in TypeScript. These are skills that are widely taught, including in the college's own computing programmes.

12 Deployment and operations

How the system is hosted, configured, checked and kept running.

Hosting

The application runs on **Vercel**; the production build runs `prisma generate` and `next build`. Live pilot: `smart-exam-timetable.vercel.app`.

Database

A **Supabase** project provides PostgreSQL and sign-in. Database migrations are applied with `npm run db:migrate:prod`.

Quality checks

GitHub Actions runs linting, type-checking, the automated unit tests and a production build on every change.

Environment settings (`.env.local`)

Setting	Purpose
`NEXT_PUBLIC_SUPABASE_URL`	Address of the Supabase project.
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Public key used by browsers for sign-in; row-level security keeps the tables private.
`SUPABASE_SERVICE_ROLE_KEY`	Server-only key for managing sign-in accounts; never sent to browsers.
`DATABASE_URL`	Database connection through Supabase's transaction pooler (port 6543), used by the running app.
`DIRECT_URL`	Direct / session connection (port 5432), used for migrations.
`ADMIN_SETUP_SECRET`	Secret for the one-time first-admin setup.
`SEED_ADMIN_EMAIL, SEED_ADMIN_PASSWORD`	Optional: when set, the seed script creates the first super-admin sign-in.

Getting started

Requirements: Node.js 20.19+, 22.12+ or 24+, and a Supabase project.

`npm ci`	install dependencies
`npm run db:generate`	generate the database client
`npm run db:migrate:prod`	apply migrations
`npm run db:seed`	load reference + demo data
`npm run dev`	run at localhost:3000
`npm test`	automated unit tests
`npm run lint`	code style checks
`npm run build`	production build
`npm run db:studio`	browse the database

Unit tests in `tests/` cover the exam-slot planner, conflict graph and scheduler, hall packing, matric numbers, session names and CSV import.

The first admin

Either set `SEED_ADMIN_PASSWORD` before seeding, which creates `admin@yabatech-examportal.com` (or `SEED_ADMIN_EMAIL`) as super admin, or call `POST /api/admin/setup` with `ADMIN_SETUP_SECRET`, which works only until the first admin exists.

Pre-loaded data

- 8 schools: SOE, SADP, SOS, SOT, SES, SLS, SMBS, STE.
- 47 departments, each with ND and HND programmes and four levels (94 programmes, 188 levels).
- About 1,000 course definitions.
- 2 verified exam halls: MPHA (200 seats) and BCLH (80); an optional script, `prisma/seed-halls.sql`, adds 11 more buildings with 2,520 seats.
- A small campus map graph.
- 50 demonstration students in Computer Technology (15 ND 1, 15 ND 2, 10 HND 1, 10 HND 2) enrolled for "2024/2025 First Semester"; sessions "2024/2025 First Semester" and "2024/2025 Second Semester".

13 Looking ahead

Opportunities to build on a successful pilot.

More departments and schools

The structure for all 8 schools and 47 departments is already loaded; rolling out is a matter of halls, sessions and student registration.

SMS and e-mail reminders

Reminders the day before each paper, with the hall and seat number.

Printable seat plans

Seat plans and attendance lists for invigilators, generated from the seat assignments.

Student records integration

Connecting to the college's student records to pre-fill registrations and course lists.

Richer campus routing

Using the campus map graph for routes along internal campus paths.

Hall-usage analytics

Insight into hall occupancy and exam-period load to plan capacity.

IN SUMMARY

YCT Exam Portal replaces days of manual timetabling with a reliable, repeatable process: clash-free schedules, hall capacity respected, a seat for every student, and live directions to the hall. It is running today for the Department of Computer Science and is ready to grow with the college.

Yaba College of Technology · YCT Exam Portal

Appendix A: Codebase guide

Where everything lives in the repository, with one line per file.

Repository: `github.com/m1r4g3-code/yct-exam-nav-system` · package name `smart-exam-timetable`. Folders in brackets, such as `(admin)`, are Next.js route groups: they organise files without adding anything to the web address.

```
yct-exam-nav-system/
├─ app/                  pages, layouts and the REST API (Next.js App Router)
│  ├─ (auth)/            sign-in, sign-up and course selection
│  ├─ (student)/         Student portal: Timetable, Navigate, Courses, Profile
│  ├─ (admin)/           Exams Office portal under /admin
│  └─ api/               REST endpoints: one route.ts per path (Appendix B)
├─ components/           admin building blocks, map, theme and the UI kit
├─ hooks/                small React hooks
├─ lib/                  auth, database and Supabase clients, shared helpers
│  └─ services/          the scheduling engine and campus routing
├─ prisma/               schema, migrations and seed data
├─ tests/                automated unit tests (npm test)
├─ scripts/              database maintenance helpers
├─ public/               crest and logo
├─ docs/                 screenshots and these presentation documents
├─ proxy.ts              access gateway: runs before every request
└─ package.json          npm scripts and dependencies
```

Figure 19. Repository map.

Where to find things

To change…	Look in
A Student portal screen	`app/(student)/…`
An Exams Office screen	`app/(admin)/admin/…`
Sign-in, sign-up or course selection	`app/(auth)/…`
An API endpoint	`app/api/<path>/route.ts`
How the timetable is generated	`lib/services/ (timetable-generator, graph-builder, exam-slots, hall-packing, hall-assigner)`
Who may access what	`proxy.ts, lib/auth.ts, app/(admin)/layout.tsx, app/(student)/layout.tsx`
The database structure	`prisma/schema.prisma + a new folder in prisma/migrations/`
Reference and demonstration data	`prisma/seed.ts`
Colours, fonts and themes	`app/globals.css`
Shared building blocks	`components/ui/, components/admin/`
The map	`components/map/`
Tests and continuous integration	`tests/, .github/workflows/ci.yml`

Root and configuration

File	Purpose
`proxy.ts`	The access gateway: runs before every page and API request, refreshes the sign-in session and admits, redirects or refuses by role.
`package.json`	App name, npm scripts (dev, build, test, lint, database commands) and dependencies.
`next.config.ts`	Next.js configuration.
`tsconfig.json`	TypeScript compiler settings (strict mode).
`eslint.config.mjs`	Linting rules (ESLint 9 flat configuration).
`postcss.config.mjs`	Connects Tailwind CSS 4 to the CSS build.
`components.json`	Settings for the shadcn/ui component generator.
`prisma.config.ts`	Prisma 7 configuration: schema and migrations location and the connection used by migrations.
`next-env.d.ts`	Generated TypeScript declarations for Next.js.
`.github/workflows/ci.yml`	Continuous integration: lint, type-check, unit tests and production build on every push and pull request.
`.gitignore · .gitattributes`	Keep secrets and build output out of version control; normalise line endings.
`README.md`	Project front page: purpose, screenshots, architecture, setup and deployment.
`AGENTS.md · CLAUDE.md`	Guidance for AI coding assistants working on the repository.

app/: application shell and authentication pages

File	Purpose
`app/layout.tsx`	Root layout: fonts, global styles, page title and the shared providers (data cache, theme, notifications).
`app/page.tsx`	The site address `/`; sends visitors to sign-in.
`app/globals.css`	Site-wide theme: Tailwind CSS 4, light and dark colour tokens and the Yabatech brand green.
`app/favicon.ico`	Browser tab icon.
`app/(auth)/layout.tsx`	Centred card layout shared by sign-in, sign-up and course selection.
`app/(auth)/login/page.tsx`	Single sign-in page for everyone; sends each user to their portal (or the page they asked for).
`app/(auth)/register/page.tsx`	Student sign-up: identity, matric number and Department → Programme → Level; signs the student in.
`app/(auth)/register/courses/page.tsx`	Select Your Courses: the level's courses pre-ticked; Complete Enrollment or Skip for now.

app/(student)/: Student portal

File	Purpose
`app/(student)/layout.tsx`	Server-side check that the visitor is a signed-in student; wraps pages in the student shell.
`app/(student)/student-shell.tsx`	Top bar on computers and bottom tab bar on phones, with theme switch and sign-out.

File	Purpose
app/(student)/dashboard/page.tsx	Exam Timetable: session selector, progress, Up Next with countdown, Upcoming and Completed.
app/(student)/courses/page.tsx	My Courses: enrol, drop or see Locked; enrolled count and total credit units.
app/(student)/navigate/page.tsx	Directory of active exam halls with map previews and coordinates.
app/(student)/navigate/[hallId]/page.tsx	Full-screen map to one hall: live position, walking route, distance and time, Google Maps hand-off.
app/(student)/profile/page.tsx	Student details, emoji avatar, theme switch and sign-out.

app/(admin)/: Exams Office portal

File	Purpose
app/(admin)/layout.tsx	Server-side check for the admin or super-admin role; wraps pages in the admin shell.
app/(admin)/admin-shell.tsx	Sidebar with crest and menu, header with page name and theme switch, footer with account and Sign out.
app/(admin)/admin/dashboard/page.tsx	Greeting, date, four headline figures and quick actions.
app/(admin)/admin/schools/page.tsx	Add, edit and delete schools.
app/(admin)/admin/departments/page.tsx	Departments by school; add (with ND/HND and four levels), edit, delete and CSV import with report.
app/(admin)/admin/programmes/page.tsx	Advanced page for viewing and editing programmes.
app/(admin)/admin/levels/page.tsx	Advanced page for viewing and editing levels.
app/(admin)/admin/courses/page.tsx	Course catalogue with cascading filters, pages of 50, add/edit/delete and CSV import.
app/(admin)/admin/halls/page.tsx	Exam halls: capacity, coordinates, description and Active status.
app/(admin)/admin/slots/page.tsx	Advanced page for viewing and adjusting exam time slots.
app/(admin)/admin/timetable/page.tsx	The command centre: sessions, Generate, report, department tabs, Edit Slot, Publish All, Seat Assignments, Reset All.
app/(admin)/admin/students/page.tsx	Student directory: search, filters, paging, Edit, Reset password and Delete.
app/(admin)/admin/students/[id]/page.tsx	One student's profile and enrolments, with Edit, Reset password and Delete.
app/(admin)/admin/students/student-dialogs.tsx	The Edit, Reset password and Delete dialogs shared by the student list and profile.

app/api/: REST API

Each endpoint is a `route.ts` file whose folder matches the web address. For example, `/api/timetable/generate` lives in `app/api/timetable/generate/route.ts`. Appendix B lists every endpoint with its methods, who may call it and its purpose.

components/

File	Purpose
components/providers.tsx	App-wide services: data cache (TanStack Query), light/dark theme and toast notifications.
components/theme-toggle.tsx	The sun/moon button that switches between light and dark mode.
components/map/DynamicMap.tsx	Loads the map only in the browser, keeping other pages light.
components/map/MapView.tsx	The interactive Leaflet map: hall pin, student position and route line, zoomed to fit.

File	Purpose
components/admin/confirm-dialog.tsx	Standard "Are you sure?" dialog before important actions.
components/admin/data-table.tsx	Reusable sortable table used by the admin lists.
components/admin/mobile-card-list.tsx	Phone-friendly card version of the admin tables.
components/admin/empty-state.tsx	Friendly placeholder for empty lists, with an optional next-step button.
components/admin/error-state.tsx	The "Couldn't load this list" panel with a *Try again* button, used on admin lists.
components/admin/import-report-dialog.tsx	Shows the result of a CSV import: imported, updated and skipped counts, and every rejected row with its reason.
components/admin/page-header.tsx	Title, subtitle and main action at the top of admin list pages.
components/admin/status-badge.tsx	The Draft / Published label on timetable rows.

components/ui/: UI kit (shadcn/ui on Base UI)

File	Purpose
badge.tsx	Small status chips (Draft, Published, Active).
button.tsx	The standard button in all its variants.
card.tsx	Bordered content cards with header, content and footer areas.
dialog.tsx	Modal windows for forms, confirmations and reports.
dropdown-menu.tsx	Row action menus (Edit, Delete, Reset password).
input.tsx · label.tsx	Text fields and their labels.
select.tsx	Drop-down pickers for sessions, schools, departments, levels and slots.
separator.tsx	Thin divider lines.
sheet.tsx	Slide-in panel used for the admin menu on phones.
sidebar.tsx	The collapsible admin sidebar (icon rail on computers, drawer on phones).
skeleton.tsx	Placeholder blocks shown while data loads.
sonner.tsx	Toast notifications matched to the app's theme.
table.tsx	Styled table elements used by every list.
tabs.tsx	Tab strips, such as the department tabs on the Timetable screen.
tooltip.tsx	Hover labels, such as page names on the collapsed sidebar.

hooks/

File	Purpose
hooks/use-selected-session.ts	Remembers the student's chosen session across the Timetable and Courses tabs.
hooks/use-user.ts	The signed-in user and role in the browser, and sign-out.
hooks/use-mobile.ts	Tells components when the screen is phone-sized.

lib/: shared server and client helpers

File	Purpose
lib/auth.ts	Identifies the caller of an API request and checks their role (admin, super admin or student).
lib/api-response.ts	One consistent response shape for every API answer: success, data, message and errors.

File	Purpose
`lib/with-handler.ts`	Wraps API handlers so unexpected errors become clear, consistent JSON responses.
`lib/fetcher.ts`	Shared helper pages use to call the API: clear error messages, and a return to sign-in when a session has expired.
`lib/matric.ts`	Matric-number normalisation and validation.
`lib/sessions.ts`	Helpers for session names, such as which semester a session is.
`lib/csv.ts`	Reads CSV files for the import screens (quoted fields, commas, line breaks).
`lib/departments.ts`	Creates a department together with its ND and HND programmes and four levels; used by the form and the CSV import.
`lib/prisma.ts`	The single shared database client.
`lib/env.ts`	Checks that the required environment settings are present.
`lib/query-keys.ts`	Central names for cached data, so screens share and refresh it consistently.
`lib/route-types.ts`	Type helper for API routes with parameters such as `[id]`.
`lib/utils.ts`	Small shared utilities, such as combining CSS class names.
`lib/supabase/client.ts`	Supabase client for the browser (sign-in and sign-out).
`lib/supabase/server.ts`	Supabase client for server code: reads the signed-in user from the request.
`lib/supabase/admin.ts`	Server-only administrative client for creating and managing sign-in accounts.

lib/services/: the scheduling engine

File	Purpose
`lib/services/exam-slots.ts`	Plans the exam slots for an exam period: weekdays, morning and afternoon.
`lib/services/slot-manager.ts`	Saves the planned slots for an exam period, reusing existing slots and keeping any added by hand.
`lib/services/graph-builder.ts`	Builds the conflict graph: courses of the same level cannot share a slot.
`lib/services/timetable-generator.ts`	The DSatur scheduler: places each course in the earliest clash-free slot with enough hall space.
`lib/services/hall-packing.ts`	Chooses halls for a class: best fit, one course per hall per slot.
`lib/services/hall-assigner.ts`	Books halls for every scheduled exam and numbers each student's seat, ready to save in one transaction.
`lib/services/dijkstra.ts`	Shortest walking path over the campus map graph (Dijkstra's algorithm).

prisma/: database

File or folder	Purpose
`prisma/schema.prisma`	The database blueprint: all 17 tables, their columns, links and enums.
`prisma/seed.ts`	Loads schools, departments, programmes, levels, courses, halls, the campus map graph, sessions and demonstration students (`npm run db:seed`).
`prisma/seed-halls.sql`	Optional SQL that adds 11 further campus buildings as exam halls (2,520 seats) with GPS coordinates.

File or folder	Purpose
`prisma/remove-stale-halls.ts`	Maintenance script that keeps the hall list to verified campus buildings.
`migrations/20260509232203_init`	Creates the original tables, enum types, unique indexes and foreign keys.
`migrations/20260510000000_add_indexes_and_constraints`	Indexes on busy columns and uniqueness rules used by the seed.
`migrations/20260515000000_add_session_check_constraints`	Database rule: session names must read "YYYY/YYYY First Semester" or "... Second Semester".
`migrations/20260515100000_add_sessions_generation_lock_drop_seatstart`	Adds the sessions and generation-lock tables and simplifies seat storage.
`migrations/20260515200000_add_indexes_and_generation_lock_fk`	Indexes for frequent queries; links each lock to a real session.
`migrations/20261014110000_add_student_id_indexes`	Speeds up look-ups of a student's enrolments and seats.
`migrations/20261014120000_enable_row_level_security`	Enables row-level security on every table.
`migrations/migration_lock.toml`	Records the database provider (PostgreSQL) for Prisma Migrate.

tests/, scripts/, public/ and docs/

File	Purpose
`tests/exam-slots.test.ts`	Exam-slot planning: weekdays, two slots a day, period length.
`tests/scheduler.test.ts`	Conflict graph and DSatur scheduler: no level clashes, hall space respected, repeatable results.
`tests/hall-packing.test.ts`	Hall choice: best fit, splitting large classes, one course per hall.
`tests/matric-and-sessions.test.ts`	Matric-number normalisation and session-name rules.
`tests/csv.test.ts`	The CSV reader used by the import screens.
`scripts/apply-migration.ts`	Maintenance helper that applies the sessions and generation-lock migration and records it in Prisma's history.
`scripts/apply-migration-indexes.ts`	Maintenance helper that applies the 20260515200000 indexes migration and records it in Prisma's history.
`public/yabatech-crest.png`	The Yabatech crest, used in the portals.
`public/yabatech-logo.png`	The Yabatech logo.
`docs/screenshots/`	Screenshots used by the README.
`docs/presentation/`	These documents: HTML sources, diagrams and the build script (`python docs/presentation/build.py`).

Appendix B: API reference

Every endpoint under `/api`, grouped by area.

All responses share one JSON shape: `{ success, data, message, errors? }`. Who can call it: Public no sign-in needed · Signed in any role · Student · Admin exams officer or super admin · Setup secret. API calls made without signing in receive a clear "please sign in again" response, and calls with the wrong role are refused.

ACCOUNTS AND SETUP

Method	Path	Who	Purpose
POST	/api/admin/setup	Setup secret	Creates the first super admin; works only until the first admin exists.
POST	/api/auth/register	Public	Student sign-up: creates the sign-in account and student record together.
GET DELETE	/api/auth	Signed in	Current user and role; sign out.
GET	/api/stats	Admin	Dashboard counts: students, courses, active halls, active sessions.

ACADEMIC STRUCTURE

Method	Path	Who	Purpose
GET POST	/api/schools	Admin	List schools; add a school.
GET PUT DELETE	/api/schools/[id]	Admin	Read, edit or delete one school.
GET POST	/api/departments	Public read Admin	List departments (used by sign-up); add a department with its ND/HND programmes and four levels.
GET PUT DELETE	/api/departments/[id]	Admin	Read, edit or delete one department.
POST	/api/departments/upload-csv	Admin	Import departments from CSV (`school_name,name,code`) with a row-by-row report.
GET POST	/api/programmes	Public read Admin	List programmes (used by sign-up); add a programme.
GET PUT DELETE	/api/programmes/[id]	Admin	Read, edit or delete one programme.
GET POST	/api/levels	Public read Admin	List levels (used by sign-up and filters); add a level.
GET PUT DELETE	/api/levels/[id]	Admin	Read, edit or delete one level.
GET POST	/api/courses	Public read Admin	The course catalogue, with filters and paging; add a course.
GET PUT DELETE	/api/courses/[id]	Admin	Read, edit or delete one course.
POST	/api/courses/upload-csv	Admin	Import or update courses from CSV with a row-by-row report.

HALLS, SLOTS AND SESSIONS

Method	Path	Who	Purpose
GET POST	/api/halls	Public read Admin	List exam halls; add a hall.
GET PUT DELETE	/api/halls/[id]	Admin	Read, edit or delete one hall.

Method	Path	Who	Purpose
GET POST	/api/slots	Admin	List exam time slots; add one by hand.
GET PUT DELETE	/api/slots/[id]	Admin	Read, edit or delete one time slot.
GET POST	/api/sessions	Public read Admin	List academic sessions (every session selector); create a session.
PATCH DELETE	/api/sessions/[id]	Admin	Make a session Active or Inactive; delete a session and its timetable.

TIMETABLE

Method	Path	Who	Purpose
POST	/api/timetable/generate	Admin	Generate the draft timetable: slots, clash-free schedule, halls and seats, plus the report.
GET	/api/timetable	Admin	A session's full timetable for review.
PATCH	/api/timetable/entry/[id]/move	Admin	Edit Slot: move one draft exam to another slot, with clash check and new halls and seats.
PUT	/api/timetable/[id]/publish	Admin	Publish All for a session ([id] is the session name), after completeness checks.
DELETE	/api/timetable/[id]/reset	Admin	Reset All: clear a session's timetable ([id] is the session name) so it can be generated again.
GET	/api/timetable/me	Student	The signed-in student's published exams with hall and seat.
GET	/api/assignments	Signed in	Seat assignments: every student's for admins, the caller's own for students.

STUDENTS AND ENROLMENTS

Method	Path	Who	Purpose
GET	/api/students	Admin	Student directory with search, filters and paging.
GET PUT DELETE	/api/students/[id]	Admin	One student with enrolments; correct their record; remove the account.
POST	/api/students/[id]/reset-password	Admin	Set a new password for a student.
GET	/api/students/me	Student	The signed-in student's own profile.
GET POST	/api/enrollments	Student	The student's enrolments for a session (with Locked status); enrol in a course (late enrolments get a free seat when possible).
DELETE	/api/enrollments/[courseId]	Student	Drop a course for a session (before publication).

NAVIGATION

Method	Path	Who	Purpose
GET	/api/navigation/halls	Public	Active halls with coordinates for the Navigate directory.
GET	/api/navigation/halls/[id]	Public	Details of one hall for the map.
GET	/api/navigation/nodes	Public	Points of the campus map graph.
GET	/api/navigation/path	Public	Shortest path between two campus map points (Dijkstra).

Appendix C: Glossary

Terms used in this document and in the app.

Session
An academic session and semester, named like "2025/2026 First Semester". Each can be Active (visible to students) or Inactive.

Semester
First or Second. Every course belongs to one; the session name decides which courses are timetabled.

Level
A year group within a programme: ND 1, ND 2, HND 1 or HND 2. Courses and students belong to a level.

Programme
A diploma track within a department: National Diploma (ND) or Higher National Diploma (HND).

Time slot
A date with a start and end time. The engine uses Morning 08:00 to 10:00 and Afternoon 12:00 to 14:00, Monday to Friday.

Timetable entry
One course's exam placed in a time slot for a session, with its halls; Draft or Published.

Draft / Published
A Draft is visible to the exams office only; Published timetables are visible to students.

Headcount
The class size used for seating: students enrolled in the course, or students registered at the level if nobody has enrolled yet.

Conflict graph
A network in which each course is a point and courses of the same level are joined, meaning they cannot share a slot.

DSatur
"Degree of saturation", a well-known graph-colouring algorithm that always places the most constrained course next. Each time slot is a colour.

Best fit
Giving a class the smallest free hall that can seat all of it, keeping larger halls free for larger classes.

Hall Overflow
A report list of courses for which there was not enough hall space; the remedy is to add halls or capacity and generate again.

Unresolved
A report list of courses for which no clash-free slot was left; the remedy is a longer exam period.

Matric number
A student's college registration number, e.g. F/ND/24/3210001, stored in capitals and unique regardless of letter case.

RLS (row-level security)
A database feature that controls access to every table; here it keeps all tables closed to browsers, so only the server can read or write data.

API
Application Programming Interface: the set of web addresses under /api that the screens use to read and change data.